

# **docker**

## **Prácticas — parte 1: fundamentos de Docker**

## Antes de empezar

- Puedes usar el entorno en la nube propuesto (tipo [Play with Docker](#)) o tu propio Docker Desktop/VM Linux/WSL2.
- No hace falta cuenta de Docker Hub para nada de lo que haremos hoy.
- Hoy hacemos los **Ejercicios 1 a 4** (miércoles) y seguimos con el resto el **viernes**, antes del cuestionario.

# Ejercicios guiados

## Ejercicio 1 — Tu primer contenedor

1. Comprueba que Docker está instalado y funcionando.
2. Ejecuta `docker run hello-world`.
3. Lee con atención el mensaje que imprime: explica qué pasos ha dado Docker por detrás.

**Resultado esperado:** entiendes, con tus palabras, el flujo "busca la imagen localmente → si no está, la descarga → crea el contenedor → lo ejecuta".

## Ejercicio 2 — Explora una imagen interactiva

1. Ejecuta `docker run -it ubuntu:22.04 bash`.
2. Dentro del contenedor, comprueba la versión del sistema ( `cat /etc/os-release` ) y crea un fichero de prueba.
3. Sal del contenedor con `exit`.
4. Ejecuta `docker ps -a` y localiza ese contenedor: ¿en qué estado está?

**Resultado esperado:** `docker ps -a` muestra el contenedor como "Exited".

## Ejercicio 3 — Contenedores de un solo uso

1. Ejecuta `docker run --rm -it alpine:3.19 sh` y dentro escribe algo en un fichero.
2. Sal con `exit`.
3. Ejecuta `docker ps -a` de nuevo.

**Resultado esperado:** el contenedor ya NO aparece en `docker ps -a` (porque `--rm` lo elimina automáticamente al salir).

## Ejercicio 4 — Levanta un servidor web

1. Ejecuta `docker run -d --name miweb -p 8080:80 nginx:1.25`.
2. Abre `http://localhost:8080` (o la URL equivalente en tu entorno) en el navegador.
3. Comprueba con `docker ps` que `miweb` está "Up" y en qué puerto.
4. Revisa sus logs con `docker logs miweb`.

**Resultado esperado:** ves la página de bienvenida de nginx en el navegador.

## Ejercicio 5 — Entra en un contenedor vivo

1. Con `miweb` todavía en ejecución, entra dentro con `docker exec -it miweb bash`.
2. Navega hasta `/usr/share/nginx/html` y mira qué ficheros hay.
3. Modifica `index.html` desde dentro del contenedor (p. ej. cambia el texto).
4. Sal con `exit` y recarga la página en el navegador.

**Resultado esperado:** el cambio se refleja en el navegador sin reiniciar el contenedor.

## Ejercicio 6 — Ciclo de vida completo

1. Para `miweb` con `docker stop miweb`. Comprueba su estado con `docker ps -a`.
2. Vuelve a arrancarlo con `docker start miweb` (sin crear uno nuevo). Comprueba que sigue respondiendo en el navegador.
3. Párelo y elimínalo definitivamente con `docker rm`.
4. Elimina también la imagen `nginx:1.25` con `docker rmi` y vuelve a descargarla con `docker pull`.

**Resultado esperado:** sabes diferenciar `stop` / `start` / `rm` y `pull` / `rmi`, y cuándo se descarga realmente una imagen.

# Retos

*Aquí no hay pasos: piensa qué comandos necesitas.*

## Reto 1 — Dos webs a la vez

**Objetivo:** levanta dos contenedores `nginx:1.25` distintos, con nombres distintos, publicados en dos puertos distintos del host, y demuestra que ambos responden a la vez en el navegador.

## Reto 2 — El contenedor que se cae solo

**Objetivo:** ejecuta un contenedor que termine inmediatamente (por ejemplo, uno que no tenga ningún proceso en primer plano), identifica en qué estado queda con `docker ps -a` y averigua, mirando sus logs, por qué ha terminado.

## Reto 3 — Limpieza general

**Objetivo:** deja tu entorno Docker completamente limpio (sin contenedores parados, sin imágenes de prueba que no vayas a volver a usar), usando los comandos adecuados en vez de borrarlos uno a uno a mano.

## Reto 4 — Diagnóstico a ciegas

Un compañero te deja un contenedor en ejecución con un nombre que no conoces y no te dice nada más.

**Objetivo:** averigua qué imagen usa, qué puertos tiene publicados y qué está escribiendo en sus logs, sin que nadie te diga el nombre del contenedor (localízalo tú con `docker ps` ).

# Autoevaluación

## ¿Te ves capaz de...?

- [ ] Explicar con tus palabras la diferencia entre una imagen y un contenedor.
- [ ] Explicar la diferencia entre un contenedor y una máquina virtual.
- [ ] Ejecutar un contenedor en segundo plano y publicando un puerto.
- [ ] Ver el estado de todos los contenedores (en ejecución y parados).
- [ ] Revisar los logs de un contenedor para diagnosticar un problema.
- [ ] Entrar con una sesión interactiva a un contenedor ya en ejecución.
- [ ] Diferenciar `docker stop / start / rm` y `docker pull / rmi`.